

团队分工与职责说明（Task Division）

本项目由四人团队协作开发。在研发过程中，由组长李伟统一进行技术架构设计与任务统筹，各组员分工明确、紧密配合，实现全链路的快速交付。团队四位成员均深度参与并为本项目付出了核心心血，共同促成了百万级企业级高可靠 RAG 检索系统的成功落地。

一、 团队角色与核心贡献

成员	职责分工	核心工作职责与输出物
李伟（组长） [组长 / 架构师]	项目统筹 & RAG 架构师	1. 系统架构设计与卸载规划：设计了本地边缘端与云端算力卸载相结合的分布式 RAG 流水线架构，实现 100 万级规模文档的批量安全建库。 2. 核心工程实现：重构并深度优化本地向量数据库管理模块 <code>embed_store.py</code>；编写 <code>preprocess.py</code> 中的自研四层语义分块算法与极短文档兜底聚拢策略。 3. 技术攻坚：定位并攻克 Mock 绑定路径穿透引起的单测报错隐患，将 Pytest 测试套件全面升级并顺利通过 82 个全量测试用例；统筹项目最终交付与远程代码库同步。
张杰（组员） [数据工程师]	数据工程师	1. 多源语料采集：编写 <code>collect_corpus.py</code>、<code>collect_stackoverflow.py</code>、<code>collect_csdn.py</code> 等脚本，实现 Wikipedia、Stack Overflow 和 CSDN 的 API 离线 Markdown 提取与本地化存储。 2. 数据清洗与高并发：设计 <code>clean_text()</code> 流水线过滤 HTML 与控制符；设计 32 线程并发元数据 LLM 提取及 429 频率超限指数退避重试算法。 3. 算力卸载部署：负责 AutoDL 远程云显卡 RTX 4090 环境搭建，编写并部署云端 FastAPI 向量计算服务器 <code>embedding_server.py</code> 与配置脚本 <code>setup_autodl.sh</code>。
王婷（组员） [前端开发工程师]	前端开发工程师	1. Web UI/UX 现代设计：采用 Vanilla CSS + 现代视觉规范重新设计 Streamlit 前端（<code>app/streamlit_app.py</code> 与 <code>app/style.css</code>），实现可折叠毛玻璃卡片和精美的“玻璃盒后端技术剖析”调试分析看板。 2. 交互逻辑开发：维护多轮对话历史、Top-K 数量及相似度阈值滑块，编写核心“数据管理”模块，支持用户拖拽上传自定义文本入库并进行异步清洗表征。 3. 安全注入防御：在前台数据渲染环节强引入 Python 标准 <code>html.escape()</code> 进行全链路字符安全过滤与实体逃逸转义，彻底阻断了由于外部语料注入而导致的 XSS 跨站脚本安全隐患。
刘洋（组员） [测试与文档工程师]	测试与文档工程师	1. 单元与集成测试：使用 <code>pytest</code> 编写覆盖全链路的单测用例（共 19 类 82 个），包含 YAML 嵌套解析、API 降级和 System 信号穿透等极端边缘用例的断言验证。 2. 答辩演示与文档：负责答辩 PowerPoint 制作（<code>BigData_RAG_答辩演示.pptx</code>）、编写演示脚本与故障应对剧本（<code>presentation_guide.md</code>）以及技术总报告（<code>report.md</code>）的撰写与行数统计维护。 3. 极速同步容灾：负责将 4.79 GB 物理 HNSW 索引打包压缩并通过 AliyunDrive 以 14MB/s 上行速度高速同步，制定一键幂等重建的防损毁数据容灾策略。

二、 协作开发流水线流程

在组长李伟的统筹指挥下，四人团队实现了高效的多任务并行，项目总计历经 81 项细节调优与安全加固，并最终在 82 个单元与集成测试中实现了 100% 成功通过 (100% Passed)，系统质量与文档质量均达到优秀的工业级交付标准。

图 2：团队协作开发流水线流程时间轴



三、 各成员负责版块技术汇报

组长：李伟 — 系统架构与向量检索核心版块汇报

作为项目组长和 RAG 系统架构师，我全面负责了本项目的整体顶层设计、核心算法模块编写以及疑难技术攻坚，确保项目架构的绝对鲁棒性与开发周期推进：

- 顶层系统架构设计与资源规划：设计了本地与云端算力卸载相结合的分布式 RAG 客服架构，确保数据在 Ingest -> Preprocess -> Embed -> ChromaDB -> Query Parser -> Hybrid Search -> QA 的全生命周期中拥有清晰的流向和严格的异常边界限制。面对 100 万级规模的数据处理，主导规划了基于 AutoDL 云端显卡（NVIDIA RTX 4090）的高效算力卸载路线，移除了本地机器算力天花板限制。
- 核心模块重构与优化 (src/embed_store.py)：重构并优化了 vector_store 向量库管理核心类。将原始的简易新增改写为支持 upsert 的幂等设计，彻底解决了大规模数据多批次重复导入引起的 DuplicateIDError 冲突。设计了高维空间余弦相似度计算与元数据 SQL where 条件端联合过滤检索算法，过滤在向量引擎层进行，有效缩减内存负荷，引入 max_distance 阈值，剔除低相关噪音文本块。
- 语义分块算法研发 (src/preprocess.py)：编写了高内聚的四级退避分块逻辑（段落 -> 句子 -> 贪心拼接 -> 滑动切割）。优先保护段落整体，在段落过长时使用标点符号识别并切割，并在相邻块间留有 120 字符的交叠 (Overlap)，最大限度保持上下文的连贯性。针对字数过短的散落文本设计了安全聚拢拼合机制，防止了语义被

物理字数粗暴切断引起的向量语义退化。

- 系统级中断信号防护与代码加固：针对大模型 API 遭遇网络超时或 429 限流报错，在代码中进行了优雅的多层捕获防崩。加入符合 AGENTS.md 规范的 KeyboardInterrupt 与 SystemExit 系统信号二次抛出防护，杜绝了底层模块静默吞下系统中断信号的问题，保障了终端的高敏捷交互。

数据工程师：张杰 — 多源语料采集与数据清洗工程版块汇报

作为数据工程师，我主要负责了本项目知识语料的动态高阶采集，超大吞吐量下的并发清洗预处理引擎，以及云端算力卸载的实际落地部署：

- 高阶多源爬虫与转换器设计：分别针对 Wikipedia、Stack Overflow 和 CSDN 博客编写了定制化的离线数据提取脚本。使用 requests 请求数据并采用 BeautifulSoup 提取核心文章 HTML DOM 节点，精准剔除了多余的侧边栏、推荐广告、脚本和页眉页脚。配合 html2text 引擎，将不同来源 of 的超文本自动转化为格式极为纯净的 Markdown 结构。
- 高并发 LLM 元数据生成与自适应退避 (src/preprocess.py)：开发了基于 ThreadPoolExecutor 的 32 线程并发元数据生成器，使整套原始文档元数据提取速度缩短了 90% 以上。面对高并发下 API 极其高发的 HTTP 429 Rate Limit（限流），在代码内实现了一套带有随机抖动的指数退避自适应重试机制（退避阶梯：2s -> 4s -> 8s，最多重试 3 次），大幅强化了数据预处理流水线在弱网环境下的韧性。
- 算力卸载与云服务器部署 (scripts/)：编写了一键配置脚本 scripts/setup_autodl.sh，在 AutoDL 云服务器上快速拉起 RTX 4090 (24GB VRAM) 的 Python 推理环境。部署了基于 FastAPI 构建的远程高性能嵌入服务 scripts/embedding_server.py，支持多进程异步推理，使用 batch_size=256 稳定消化百万级数据，实现吞吐量 185 句/秒，耗时 2.5 小时以极其低廉的租用成本（4.70元）完成了全量高维向量表征提取。

前端开发工程师：王婷 — 前端现代美学交互与安全版块汇报

作为前端开发工程师，我专注于为智能客服系统构建现代、友好且具备深度交互与调试分析能力的可视化 Web 前端界面，并构筑前台数据的防注入安全屏障：

- Vanilla CSS 现代美学视觉重塑 (app/style.css)：基于当下主流的 Glassmorphism（毛玻璃）与流畅的深色卡片布局规范设计了全套视觉层。重新编写了全局滚动条、可展开调节面板与输入文本框的拟态拟物过渡动画，让 Streamlit 这个原本默认布局刻板的框架呈现出极富质感的交互界面。
- 多轮对话状态机与“玻璃盒”剖析看板开发：借助 Streamlit 的 session_state 构建了支持上下文追踪的多轮对话状态机，支持用户即席选择并调整 Top-K 检索数量和相似度过滤分值（余弦距离阈值）。在主界面下方设计了能够实时一键展开的“后端技术剖析面板”，清晰展示系统检索出的各参考文档的余弦得分、文档分类、年份，方便技术人员直观分析检索精度。
- 安全注入防御与转义网关：在接收到检索答案并向浏览器前端渲染之前，在变量输出入口强引入了 Python 标准 html.escape() 进行字符安全过滤与字符实体逃逸转义，确保外源文本（例如带有漏洞注入指令的 Wikipedia 词条）无法在前台执行恶意 XSS 挂马，保障浏览器沙箱安全。

测试与文档工程师：刘洋 — Pytest 测试套件开发与技术文档保障版块汇报

作为测试与文档工程师，我全力确保本项目所有模块的工程质量达到生产环境的验收指标，并完成了全套答辩与技术文字资产的整理归纳：

- 基于 Mock 的 Pytest 全覆盖测试设计 (tests/)：构建了涵盖所有核心组件的自动化测试包，包含 19 个独立测试类，共 82 个详尽的单元测试与端到端集成测试用例，覆盖率为团队开发之最。全面采用了 unittest.mock.patch 装饰器对外部 OpenAI 接口和 Hugging Face Embedding 下载环境实施无物理依赖的完美模拟，使得开发环境和测试管道可在无网状态下瞬时运行全套测试。
- 总技术报告及答辩 PowerPoint 制作：撰写了深入的技术报告 report/report.md，从算法的底层角度剖析防幻觉 Prompt（如在检索无匹配时严格返回“参考资料不足”并不自行发散）的几度迭代，并在报告中更新并对齐最新的数据口径。制作了 BigData_RAG_答辩演示.pptx，以优雅的图表与极具逻辑的技术架构拆解幻灯片。
- 物理索引打包与数据容灾策略：制定了 HNSW 数据库备份容灾策略。由于百万级建库需要 2.5 小时，将生成的 4.79 GB 物理 HNSW 数据库使用 tar.gz 极速压缩打包，通过网盘极速同步（14MB/s）同步至本地，使得团队成员本地可以实现零成本、即刻恢复运行，消除了数据丢失与重构的高昂算力开销。